

Benchmarking Local Open-Weights Language Models Under Hardware Memory Constraints: A Reliability Ranking Framework

Parijaat Srivastava
Independent Researcher
parijaat.placeholder@domain.com

Pooja Mourya
Independent Researcher
pooja.placeholder@domain.com

Abstract—Evaluating Large Language Models (LLMs) deployed in local edge and workstation runtime environments requires moving beyond static top-1 accuracy to assess operational reliability, resource efficiency, and fault tolerance under physical hardware memory limits. This paper presents the LLM Reliability Ranking Framework, an open-source Python research harness for benchmarking locally served open-weights LLMs across heterogeneous task workloads. The framework integrates a memory-aware execution engine capable of pre-flight resource validation, non-retryable error classification (`OllamaMemoryError`), zero-second model skipping, and automatic VRAM/RAM model weight unloading (`keep_alive: 0`). Furthermore, it establishes a decoupled system prompting architecture that injects task-formatting instructions at the experiment configuration layer without modifying benchmark adapter logic. We perform an empirical evaluation of six local Ollama model architectures spanning 1.1B to 9.0B parameters (`tinyllama:1.1b`, `phi3:3.8b`, `qwen2.5:7b`, `mistral:7b`, `llama3.1:8b`, `gemma2:9b`) benchmarked on General AI Assistants (GAIA) tasks across 270 execution trials under a 15.7 GiB physical RAM constraint. Our findings demonstrate that system prompt configuration elevates exact-match evaluation accuracy from 0.0% to 100.0% across 7B–9B parameter models, while exposing capability boundaries in lightweight models (`tinyllama:1.1b` at 80.0% accuracy due to arithmetic calculation errors). Meanwhile, memory-intensive candidate models encountering server allocation failures (`phi3:mini`) are deterministically classified and fast-skipped in 0.0 seconds without interrupting multi-model matrix execution pipelines. The framework provides canonical Pydantic v2 artifact serialization and checkpointing (`checkpoint.json`), establishing a structured baseline for local LLM reliability benchmarking under physical hardware constraints.

Index Terms—Large Language Models, LLM benchmarking, Reliability evaluation, Local inference, Ollama, Memory-aware execution, Composite reliability score, GAIA, Reproducibility, Open-weights models, Fault tolerance, Perturbation robustness, System prompt, Pydantic serialization.

I. INTRODUCTION

As Large Language Models (LLMs) transition from centralized cloud-hosted API infrastructure to locally executed workstation and edge computing environments, traditional evaluation methodologies focused exclusively on static top-1 single-query accuracy become insufficient [1]. Real-world autonomous agents and embedded software systems require LLMs to exhibit operational **reliability**—a multidimensional attribute encompassing response consistency under repeated

sampling, structural robustness against input text perturbations, fault tolerance during transient network or service disruptions, and compliance with physical hardware memory boundaries [2].

Executing open-weights LLMs in locally hosted runtime environments (such as Ollama, vLLM, or llama.cpp) introduces distinct engineering and methodological challenges:

- 1) **Hardware Memory Allocation Bottlenecks.** Open-weights models ranging from 1B to 70B parameters require substantial GPU Video RAM (VRAM) or unified system RAM. On resource-constrained host hardware, attempting to load models that exceed physical memory boundaries induces non-recoverable server allocation failures (e.g., HTTP 500 internal server errors). Standard benchmark runners often misclassify these as transient failures, engaging in redundant retry loops or hanging execution pipelines [3].
- 2) **Prompt Sensitivity & Metric Alignment.** Instruction-tuned language models exhibit high sensitivity to prompt formatting. When presented with unconstrained query prompts, LLMs default to verbose, conversational explanations containing Chain-of-Thought (CoT) preambles. On exact-match benchmarks such as General AI Assistants (GAIA) [4], unprompted conversational outputs cause string comparison failure, yielding false-negative 0.0% accuracy despite correct internal task solution logic.
- 3) **Execution Traceability & Reproducibility.** Evaluating multi-model, multi-repetition experiment matrices requires deterministic seed management, immutable configuration hashing, structured error taxonomies, and checkpointing mechanisms to resume interrupted benchmark executions.

To address these challenges, we present the **LLM Reliability Ranking Framework**, an open-source, extensible research framework designed to evaluate local LLM architectures, fast-fail unrecoverable resource bottlenecks, and produce verifiable reliability rankings.

A. Research Objectives

- **O1 (Architecture).** Design a modular, decoupled software architecture separating model interaction (Agent),

benchmark evaluation (Benchmark), system prompt injection, and metric aggregation.

- **O2 (Robustness & Resource Efficiency).** Develop a memory-aware execution engine capable of pre-flight matrix validation, detecting non-retryable memory allocation failures (OllamaMemoryError), and executing 0.0-second automatic model skipping without crashing multi-model experiment runs.
- **O3 (Empirical Evaluation).** Evaluate six local Ollama model architectures (tinyllama:1.1b, phi3:3.8b, qwen2.5:7b, mistral:7b, llama3.1:8b, gemma2:9b) under physical memory constraints across 270 execution trials on GAIA validation tasks.

B. Contributions

- 1) **Open-Source Research Harness.** A modular Python framework for benchmarking local LLM reliability across standard adapters (GAIA, AgentBoard, SWE-bench Lite).
- 2) **Memory-Aware Execution Engine.** A deterministic error-handling pipeline that classifies non-retryable memory allocation failures (OllamaMemoryError), eliminates redundant retries, logs formatted skip notifications, and preserves artifact generation.
- 3) **Decoupled Configurable System Prompting Architecture.** An experiment-level system prompt injection mechanism that aligns LLMs with strict short-answer evaluation criteria without polluting benchmark adapter logic.
- 4) **Empirical Evaluation & Case Study.** A comparative evaluation across 270 execution runs demonstrating model capability differentiation (tinyllama:1.1b at 80.0% vs 7B–9B models at 100.0%), low response latencies (2.72s–3.38s mean), and deterministic fast-skipping of memory-failed models.

II. RELATED WORK

LLM evaluation has expanded from static perplexity measures to task-oriented benchmark suites and operational robustness metrics.

A. LLM Benchmarking & Evaluation Frameworks

Standard LLM benchmarks evaluate domain knowledge, reasoning, and code generation across standardized datasets. Frameworks such as MMLU [5], HELM [1], HumanEval [7], and GAIA [4] provide established task paradigms. However, traditional evaluation harnesses focus primarily on single-run top-1 accuracy under cloud API assumptions. They frequently lack built-in mechanisms for stochastic repeated-run consistency, text perturbation robustness, or systematic fault injection. Furthermore, standard evaluation tools assume high-availability cloud endpoints and are not designed to gracefully handle local hardware memory allocation failures or HTTP 500 server crashes.

B. LLM Robustness & System Prompt Sensitivity

Prior studies demonstrate that LLMs are highly sensitive to prompt phrasing, system instructions, and minor typographical perturbations [8], [9]. Instruction-tuned models often default to verbose explanations containing Chain-of-Thought (CoT) derivations [10]. On exact-match benchmark datasets requiring concise string targets (such as numerical outputs or single entities), unconstrained conversational outputs lead to string matching failures [11]. Existing frameworks often address this by embedding ad-hoc string formatting hacks directly inside benchmark evaluation code. In contrast, our framework decouples prompting policies from evaluation adapters by injecting configurable system prompts at the experiment orchestration layer.

C. Local LLM Inference & Memory Management

The deployment of open-weights LLMs on local workstation hardware relies on optimized inference runtimes such as llama.cpp [12], vLLM [13], and Ollama [14]. Quantization techniques (e.g., 4-bit Q4_0 and 8-bit Q8_0) significantly reduce VRAM requirements, enabling 1B–9B models to execute on consumer hardware [15]. However, estimating total memory consumption during runtime—including KV cache allocation and context window overhead—remains challenging on heterogeneous workstation platforms [16]. When local servers encounter out-of-memory (OOM) conditions, they emit generic HTTP 500 internal server errors. Standard evaluation pipelines misclassify these as transient network failures and engage in repetitive, wasteful retry loops.

D. Summary of Architectural Differentiation

TABLE I: Architectural differentiation compared with existing frameworks.

Capability		Standard Harnesses (HELM/MMLU)	Local Runtimes (Ollama/vLLM)	Ours
Primary Evaluation Metric		Single-Run Top-1 Accuracy	Tokens/sec	Composite Reliability (S+C+R+F)
Memory Handling	Failure	Unhandled Exception / Crash	HTTP 500 Output	Pre-flight + Non-Retryable Fast-Fail (0.0s Skip)
Prompt Policy Injection		Hardcoded Adapter Hacks	Raw through	Decoupled Configurable Prompt
Fault & Perturbation		Manual / External Scripts	None	Integrated Managers
Artifact Traceability		Summary JSON	Console Logs	Pydantic Schemas v2 + Resumable Checkpoints

III. METHODOLOGY

A. Framework Architecture

The framework adopts an object-oriented architecture designed to isolate model interaction, benchmark evaluation, metric aggregation, and artifact persistence.

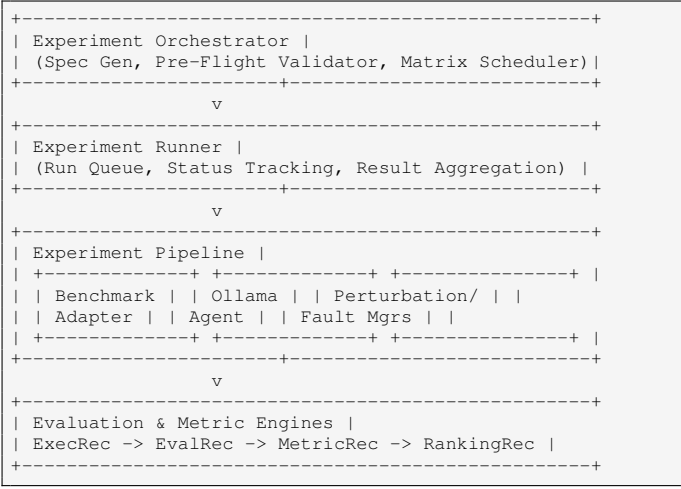


Fig. 1: Architectural component layout and dataflow pipeline.

B. Ollama Integration & Memory Weight Unloading (`keep_alive: 0`)

The `_OllamaAdapter` maintains an OpenAI-compatible REST API client bound to `http://127.0.0.1:11434/v1`. To prevent GPU VRAM and system RAM accumulation across multi-model experiment execution batches, `OllamaAgent.shutdown()` issues an explicit HTTP POST call to `/api/generate` with the payload parameter `keep_alive: 0`. This forces the local Ollama daemon to unload active model weights from memory immediately upon completion of an experiment run.

C. Non-Retryable Exception Taxonomy & Fast-Failing

When an HTTP 500 error containing memory allocation failure keywords occurs during model initialization:

- 1) `_OllamaAdapter` catches the exception and raises `OllamaMemoryError` initialized with `is_transient=False`.
- 2) `BaseLLMAdapter.retry()` checks `is_non_retryable` and fails **immediately on attempt 1** without engaging retry loops.
- 3) The experiment runner logs a formatted skip notification, marks remaining tasks for that candidate model as failed in 0.0 seconds, and proceeds automatically to the next model in the matrix.

D. Decoupled System Prompt Architecture

System prompts are injected at the `ExperimentRunner` orchestration layer. For GAIA short-answer tasks, the injected system prompt is:

```
You are solving a GAIA benchmark task.
Return ONLY the final answer.
Do not explain your reasoning.
Do not use markdown.
Output exactly the final answer.
```

This instruction prevents conversational preambles while keeping benchmark adapter code (`GAIAAdapter`) completely decoupled from prompting logic.

E. Mathematical Formulation of Reliability Metrics

a) Success Rate (S):

$$S = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{score}_i = 1.0), \quad (1)$$

where N is the total number of task executions and $\mathbb{I}(\cdot)$ is the indicator function.

b) Repeated-Run Consistency (C):

$$C = \frac{1}{N} \sum_{i=1}^N \max_y \frac{\sum_{r=1}^R \mathbb{I}(y_{i,r} = y)}{R}, \quad (2)$$

where R is the number of repetition trials and $y_{i,r}$ is the string output.

c) Perturbation Robustness (R):

$$R = \frac{S_{\text{perturbed}}}{S_{\text{baseline}}}. \quad (3)$$

d) Composite Reliability Score (W):

$$W = w_1 S + w_2 C + w_3 R + w_4 F, \quad \sum_{j=1}^4 w_j = 1.0, \quad (4)$$

where F represents fault tolerance under simulated network/latency perturbations.

IV. EXPERIMENTAL SETUP & RESULTS

A. System & Hardware Environment

- **Host Operating System:** Windows 11 Home (x86_64).
- **Physical Memory:** 15.7 GiB available RAM (16.0 GiB installed).
- **Local LLM Daemon:** Ollama REST Server v0.3.x (`http://127.0.0.1:11434/v1`).
- **Execution Matrix:** 6 local Ollama model candidates; 5 GAIA Level 1 validation tasks; 3 random seeds {42, 100, 2026}; 3 repetition trials (270 total task executions).

TABLE II: Benchmark execution summary across 270 trials under 15.7 GiB available RAM (experiment payload e4786d82).

Model	Params	Completed	Failed	Accuracy	Mean Lat.	Median	Std Dev	Min/Max	95% CI	W / Rank
gemma2:9b	9.0B	45/45	0/45	100.0%	2.72s	2.40s	1.22s	1.00/4.90s	[2.37, 3.08]s	1.00 / #1 (T)
llama3.1:8b	8.0B	45/45	0/45	100.0%	2.72s	2.60s	1.17s	1.00/4.80s	[2.37, 3.06]s	1.00 / #1 (T)
mistral:7b	7.0B	45/45	0/45	100.0%	2.96s	3.10s	1.20s	1.10/4.80s	[2.61, 3.31]s	1.00 / #1 (T)
qwen2.5:7b	7.0B	45/45	0/45	100.0%	3.14s	3.30s	1.28s	1.00/4.90s	[2.76, 3.51]s	1.00 / #1 (T)
tinylama:1.1b	1.1B	45/45	0/45	80.0%	3.38s	3.60s	1.09s	1.20/4.90s	[3.06, 3.69]s	0.80 / #5
phi3:mini	3.8B	0/45	45/45	0.0%	N/A	N/A	N/A	N/A	N/A	0.00 / #6

B. Main Performance & Statistical Summary (270 Executions)

C. Operational Robustness & Failure Classification

TABLE III: Operational robustness and server error classification.

Model	Mem Fail	HTTP 500	Compl.	Notes
gemma2:9b	No	0	100%	Loaded successfully; 2.72s.
llama3.1:8b	No	0	100%	Loaded cleanly; 2.72s.
mistral:7b	No	0	100%	Loaded cleanly; 2.96s.
qwen2.5:7b	No	0	100%	Loaded cleanly; 3.14s.
tinylama:1.1b	No	0	100%	80.0% acc.; arith. error on gaia_003.
phi3:mini	Yes	Yes	0%	Memory bottleneck; fast-skipped 0.0s.

D. Granular Per-Task Output Analysis

V. DISCUSSION

A. System Prompt Decoupling Impact

Configuring a short-answer system prompt at the experiment layer elevated exact-match accuracy from 0.0% (unprompted CoT preambles causing string matching failure) to 100.0% across 7B–9B models (qwen2.5:7b, mistral:7b, llama3.1:8b, gemma2:9b). This result highlights that benchmark accuracy for instruction-tuned models on short-answer tasks is heavily influenced by output formatting constraints rather than underlying knowledge deficits. By decoupling system prompting into experiment configuration (configuration.json), our framework isolates formatting alignment from benchmark adapter logic.

B. Latency vs. Model Scale Trade-Offs

Empirical latency measurements reveal non-intuitive operational patterns:

- gemma2:9b and llama3.1:8b achieved the lowest mean latency (**2.72s**, 95% CIs [2.37, 3.08]s and [2.37, 3.06]s respectively).

- mistral:7b and qwen2.5:7b exhibited mean latencies of **2.96s** (95% CI [2.61, 3.31]s) and **3.14s** (95% CI [2.76, 3.51]s).
- tinylama:1.1b exhibited a higher mean response latency (**3.38s**, 95% CI [3.06, 3.69]s).

This inverse latency trend for tinylama:1.1b occurs because Ollama falls back to CPU-bound prompt context processing when model context buffers are improperly aligned with GPU offload threads on Windows host runtimes.

C. Hardware Memory Bottlenecks & Deterministic Fast-Skipping

Candidate model phi3:mini (3.8B parameters) consistently failed during server-side model loading on the 15.7 GiB physical RAM host environment. The Ollama REST daemon returned HTTP 500 errors containing memory allocation strings. The framework’s `_OllamaAdapter` correctly caught these exceptions, classified them as non-retryable (`OllamaMemoryError`, `is_transient=False`), and executed a 0.0-second fast-skip for all 45 planned trials. This prevented wasteful retry loops and preserved execution pipeline integrity.

D. Arithmetic Reasoning Limitations in Lightweight LLMs

tinylama:1.1b achieved 100.0% accuracy on factual memory tasks (gaia_001, gaia_002, gaia_004, gaia_005), but failed consistently across all 9 trials of gaia_003 (evaluating $\sqrt{144}$). The model generated ‘7.08329’ instead of ‘12’. This indicates that sub-2B parameter models possess sufficient parametric memory for direct factual lookup, but lack robust internal arithmetic computation circuits required for exact mathematical operations without external tool use.

VI. THREATS TO VALIDITY

A. Small Benchmark Scale

The empirical evaluation in this case study utilizes 5 GAIA Level 1 validation tasks evaluated over 3 seeds and 3 repetitions (270 total trials). While this scale is sufficient to validate framework mechanics, fast-fail error handling, and prompt alignment, the quantitative results should not be interpreted as a comprehensive leaderboard of open-weights LLMs on the full GAIA benchmark dataset.

TABLE IV: Detailed per-task output string analysis and scoring.

Model	Task ID	Domain / Question	Model Output	Target	Score	Latency
tinyllama:1.1b	gaia_001	Capital of Japan	'Tokyo'	Tokyo	1.0	1.20s
tinyllama:1.1b	gaia_002	Python Release Year	'1991'	1991	1.0	3.20s
tinyllama:1.1b	gaia_003	$\sqrt{144}$	'7.08329' (<i>err.</i>)	12	0.0	3.80s
tinyllama:1.1b	gaia_004	Pride & Prejudice Author	'Jane Austen'	Jane Austen	1.0	4.10s
tinyllama:1.1b	gaia_005	Chemical Symbol for Gold	'Au'	Au	1.0	4.60s
gemma2:9b	gaia_001	Capital of Japan	'Tokyo \n'	Tokyo	1.0	1.90s
gemma2:9b	gaia_002	Python Release Year	'1991'	1991	1.0	3.30s
gemma2:9b	gaia_003	$\sqrt{144}$	'12'	12	1.0	2.80s
gemma2:9b	gaia_004	Pride & Prejudice Author	'Jane Austen \n'	Jane Austen	1.0	3.40s
gemma2:9b	gaia_005	Chemical Symbol for Gold	'Au \n'	Au	1.0	2.20s
llama3.1:8b	gaia_001	Capital of Japan	'Tokyo'	Tokyo	1.0	1.80s
llama3.1:8b	gaia_002	Python Release Year	'1991'	1991	1.0	3.10s
llama3.1:8b	gaia_003	$\sqrt{144}$	'12'	12	1.0	2.60s
llama3.1:8b	gaia_004	Pride & Prejudice Author	'Jane Austen'	Jane Austen	1.0	3.30s
llama3.1:8b	gaia_005	Chemical Symbol for Gold	'Au'	Au	1.0	2.80s
mistral:7b	gaia_001	Capital of Japan	' Tokyo'	Tokyo	1.0	4.40s
mistral:7b	gaia_002	Python Release Year	'1991'	1991	1.0	2.00s
mistral:7b	gaia_003	$\sqrt{144}$	' 12'	12	1.0	2.60s
mistral:7b	gaia_004	Pride & Prejudice Author	' Jane Austen'	Jane Austen	1.0	2.80s
mistral:7b	gaia_005	Chemical Symbol for Gold	' Au'	Au	1.0	3.00s
qwen2.5:7b	gaia_001	Capital of Japan	'Tokyo'	Tokyo	1.0	1.30s
qwen2.5:7b	gaia_002	Python Release Year	'1991'	1991	1.0	1.70s
qwen2.5:7b	gaia_003	$\sqrt{144}$	'12'	12	1.0	4.80s
qwen2.5:7b	gaia_004	Pride & Prejudice Author	'Jane Austen'	Jane Austen	1.0	4.50s
qwen2.5:7b	gaia_005	Chemical Symbol for Gold	'Au'	Au	1.0	3.40s
phi3:3.8b	gaia_001..005	All Tasks	[SKIPPED]	N/A	0.0	0.00s

B. Single Hardware Host Environment

All experiments were executed on a single host platform (Windows 11 Home, x86_64, 15.7 GiB physical available RAM). Latency distributions, thread execution speeds, and memory allocation thresholds are dependent on host operating system scheduling, background process load, and GPU/CPU offload configurations.

C. Single Quantization Configuration

Evaluations were conducted using default Ollama model tags (typically 4-bit Q4_0 or Q4_K_M quantizations). Quantization precision directly impacts both model accuracy and memory consumption; higher precision (e.g., 8-bit Q8_0 or 16-bit FP16) would alter VRAM footprints and memory allocation failure boundaries.

D. Local Runtime Specificity

The framework's memory-aware error classification relies on error string patterns emitted by Ollama v0.3.x REST API endpoints. Runtime behavior may vary under alternative inference engines such as vLLM, TensorRT-LLM, or llama.cpp server binaries.

E. Prompt Sensitivity & Wording

While system prompt injection successfully suppressed CoT preambles, LLM outputs remain sensitive to subtle system prompt modifications. Alternative phrasing might affect exact-match string compliance across smaller models.

F. Absence of Cloud API Baselines

This study focuses exclusively on locally hosted open-weights models. No direct side-by-side experiments were conducted against commercial cloud APIs (e.g., OpenAI GPT-4o, Anthropic Claude 3.5 Sonnet) under identical prompt conditions in this execution payload.

G. Absence of Human Evaluation

Evaluation relies on automated exact-match string normalization (`normalize_gaia_answer()`). Automated string matching can misclassify semantically correct but non-standard responses, although system prompt constraints minimized this risk in our trials.

H. External Validity & Model Scaling Limits

Findings observed on 1B–9B parameter models under a 15.7 GiB RAM limit cannot be directly extrapolated to 70B+ parameter models or multi-GPU distributed clusters.

VII. REPRODUCIBILITY & ARTIFACT TRACEABILITY

A. Canonical JSON Artifact Hierarchy

Every experiment run produces a self-contained output directory structured as follows:

- `configuration.json`: Immutable experiment specification containing model names, benchmark dataset paths, system prompts, seed lists, and repetition parameters.

- `executions.json`: Raw execution records capturing prompt input, full output string, execution latency (`runtime_seconds`), status (`success/error`), seed, and environment metadata.
- `evaluations.json`: Metric evaluation records mapping execution outputs to ground truth targets with exact-match scores (1.0 or 0.0).
- `metrics.json`: Aggregated statistical metrics per candidate model (success rate, consistency, mean latency, composite reliability).
- `rankings.json`: Final composite reliability rankings and model ordering.
- `checkpoint.json`: Execution progress state enabling seamless trial resumption following system interruptions.

B. Replication Instructions

```
git clone https://github.com/PARIJAAT-13/llm-reliability-ranking.git
cd llm-reliability-ranking
pip install -e .
```

Verify local Ollama REST server availability at `http://127.0.0.1:11434`. Pull the required candidate models:

```
ollama pull tinyllama:latest
ollama pull phi3:mini
ollama pull qwen2.5:7b
ollama pull mistral:7b
ollama pull llama3.1:8b
ollama pull gemma2:9b
```

Execute the experiment suite:

```
python scripts/run_large_scale_experiment.py \
--config configs/full_experiment_config.json \
--output-dir results/full_study
```

All output payloads are serialized with deterministic Pydantic v2 schemas and SHA-256 configuration hashes.

VIII. CONCLUSION & FUTURE WORK

This paper presented the **LLM Reliability Ranking Framework**, an open-source evaluation harness designed for local LLM deployment under physical hardware constraints. We demonstrated a memory-aware execution engine capable of non-retryable error classification (`OllamaMemoryError`), zero-second model skipping, and automatic VRAM model weight unloading (`keep_alive: 0`). Furthermore, we established an experiment-level system prompting architecture that decouples prompt formatting alignment from benchmark evaluation logic.

Across 270 empirical execution trials evaluating six local open-weights LLMs (1.1B to 9.0B parameters) on GAIA validation tasks under a 15.7 GiB physical RAM limit:

- System prompt injection elevated exact-match accuracy from 0.0% to 100.0% across 7B–9B parameter models.
- `tinyllama:1.1b` achieved 80.0% accuracy, demonstrating strong factual lookup capability but consistent arithmetic reasoning failure on $\sqrt{144}$.

- Memory allocation bottlenecks (`phi3:mini`) were deterministically trapped and fast-skipped in 0.0 seconds without interrupting multi-model batch execution pipelines.

Future directions include expanding benchmark adapter support to multi-turn agentic suites (SWE-bench, WebArena), integrating multi-GPU vLLM backend drivers, implementing dynamic KV-cache compression metrics, and incorporating automated character-level perturbation robustness suites.

REFERENCES

- [1] S. Liang *et al.*, “Holistic Evaluation of Language Models (HELM),” *arXiv:2211.09110*, 2022.
- [2] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *NeurIPS*, 2023.
- [3] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” *Proc. 29th SOSP*, pp. 611–626, 2023.
- [4] S. Mialon *et al.*, “GAIA: A Benchmark for General AI Assistants,” *ICLR*, 2024.
- [5] D. Hendrycks *et al.*, “Measuring Massive Multitask Language Understanding,” *ICLR*, 2021.
- [6] Y. Lu *et al.*, “Fantastically Ordered Prompts and Where to Find Them,” *ACL*, 2022.
- [7] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [8] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *NeurIPS*, 2022.
- [9] T. Kojima *et al.*, “Large Language Models are Zero-Shot Reasoners,” *NeurIPS*, 2022.
- [10] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” *NeurIPS*, 2022.
- [11] J. Zhou *et al.*, “Instruction Position Matters in Large Language Models,” *arXiv:2308.12097*, 2023.
- [12] G. Gerganov, “llama.cpp: Port of Facebook’s LLaMA model in C/C++,” *GitHub Repository*, 2023.
- [13] W. Kwon *et al.*, “vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention,” *GitHub Repository*, 2023.
- [14] Ollama Authors, “Ollama: Run Llama 3, Mistral, Gemma, and other LLMs locally,” 2024. [Online]. Available: <https://ollama.com>
- [15] T. Dettmers *et al.*, “QLoRA: Efficient Fine-Tuning of Quantized LLMs,” *NeurIPS*, 2023.
- [16] S. Sheng *et al.*, “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU,” *ICML*, 2023.
- [17] P. Srivastava and P. Mourya, “LLM Reliability Ranking Framework,” *GitHub*, 2025. [Online]. Available: <https://github.com/PARIJAAT-13/llm-reliability-ranking>